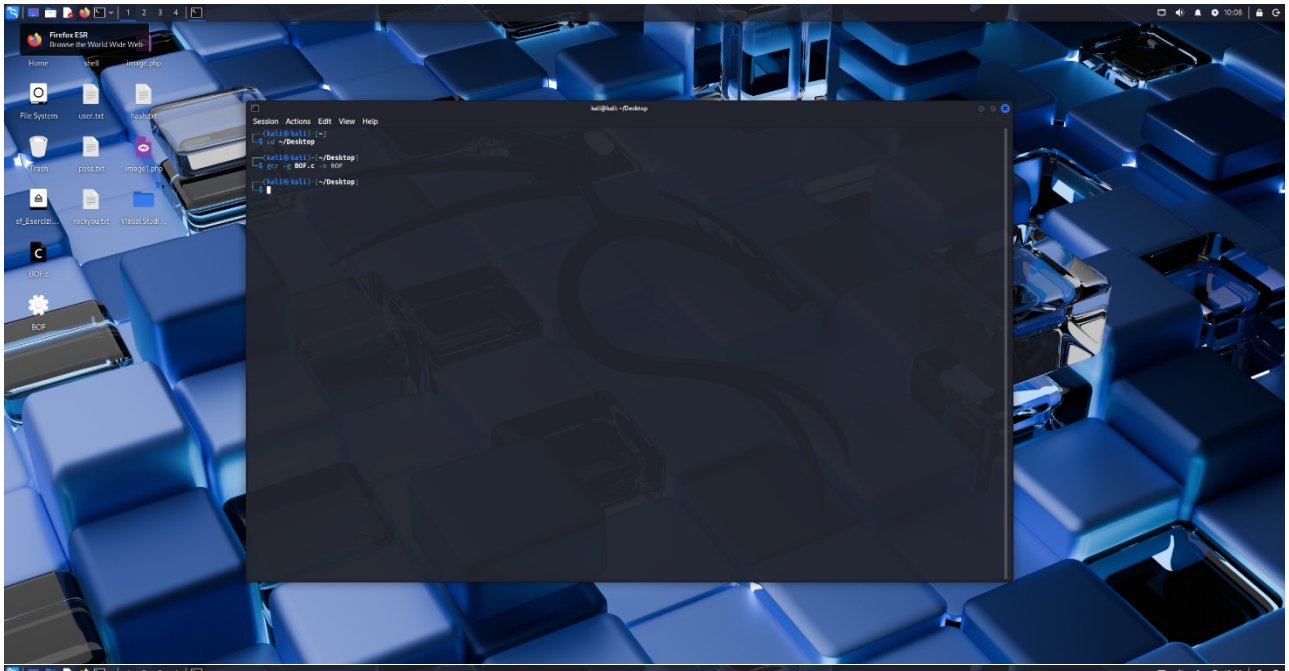
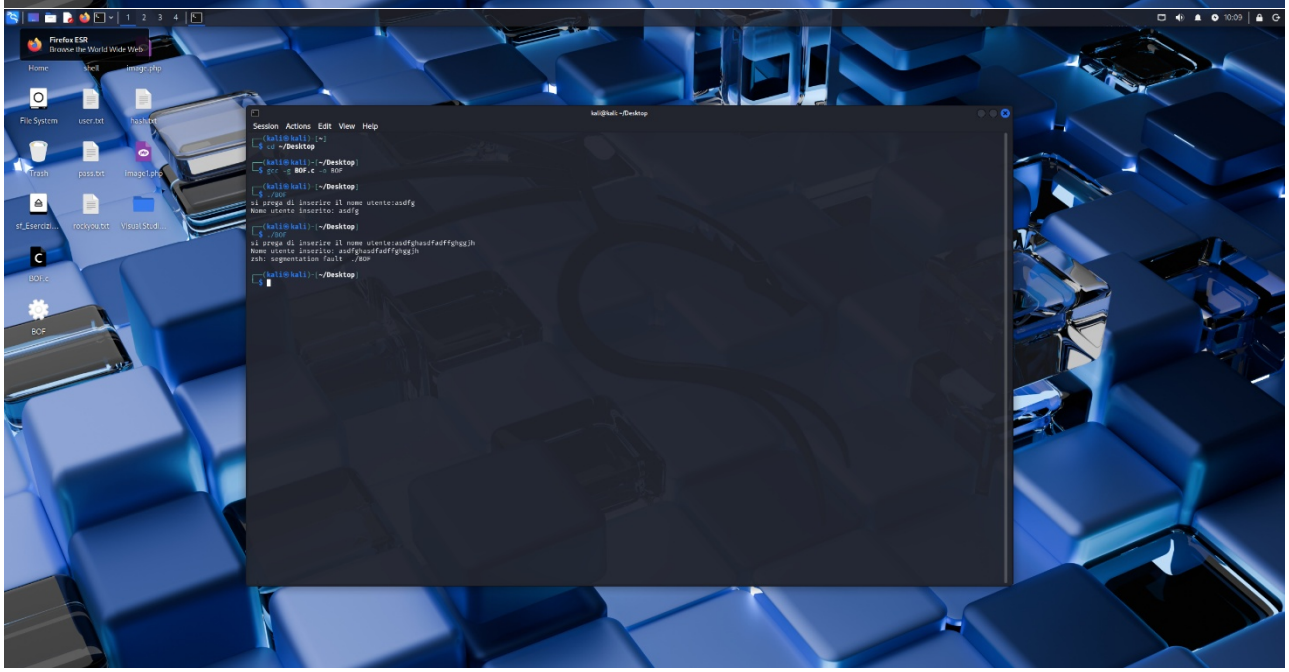
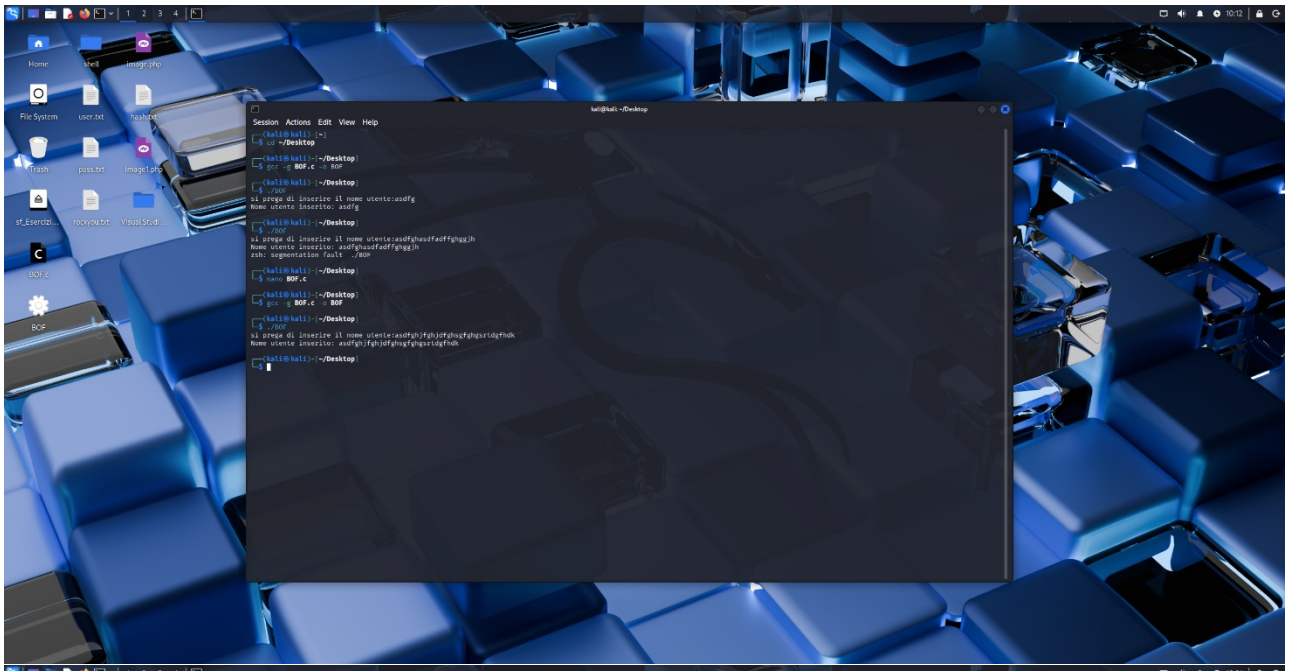
REPORT TECNICO: ANALISI DELLA VULNERABILITÀ BUFFER OVERFLOW

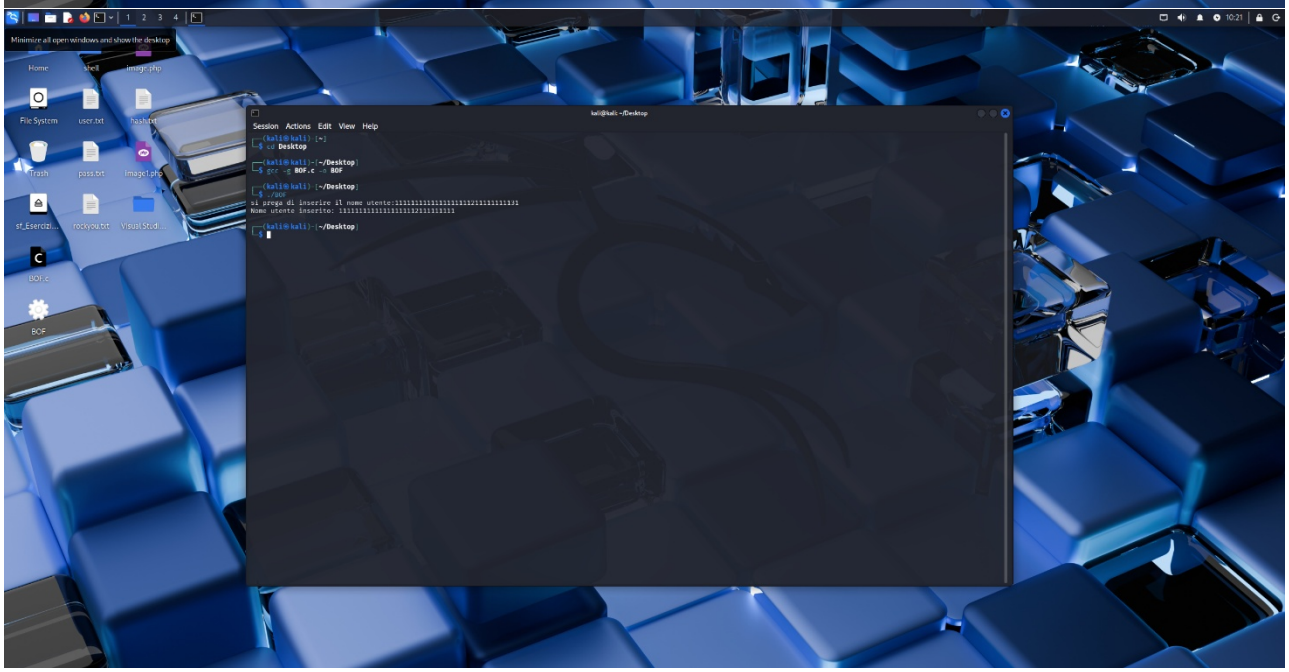
Data: 24 Aprile 2026

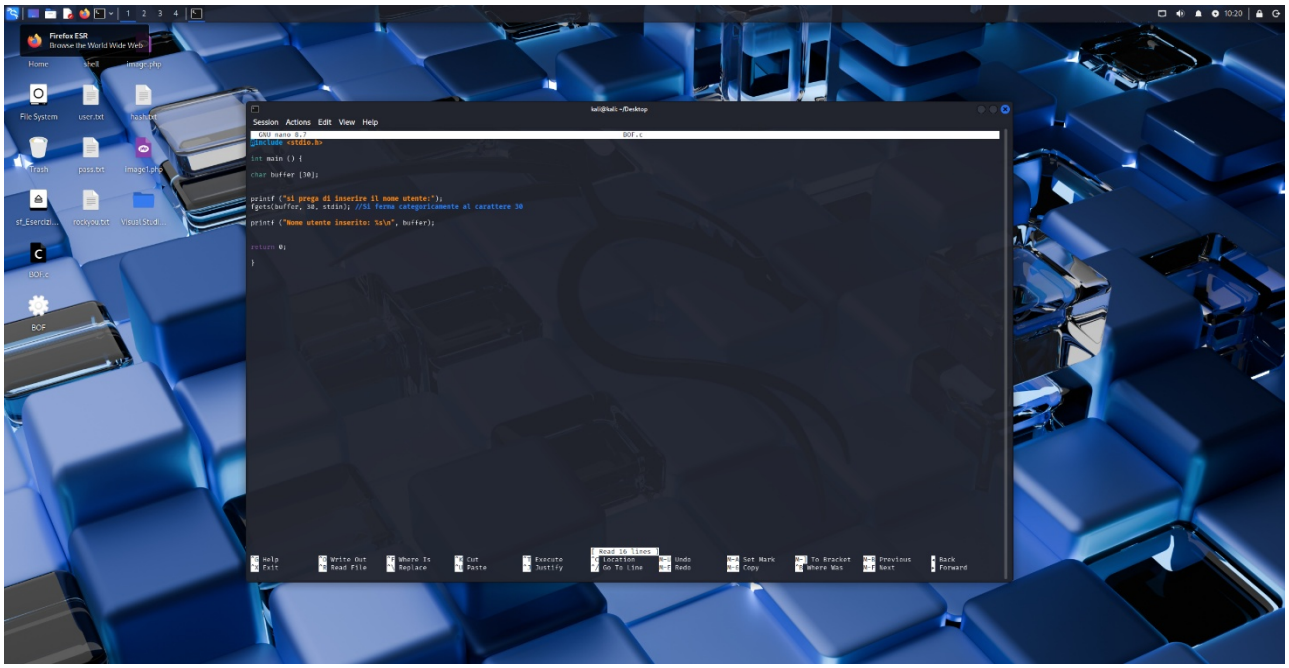
Oggetto: Valutazione della mitigazione tramite espansione del buffer nel file BOF.c






kali@kali: ~/Desktop
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ cd ~/Desktop
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
si prega di inserire il nome utente:asdfg
Nome utente inserito: asdfg
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
si prega di inserire il nome utente:asdfghasdfadffghgg jh
Nome utente inserito: asdfghasdfadffghgg jh
zsh: segmentation fault ./BOF
┌──(kali㉿kali)-[~/Desktop]
└─$ nano BOF.c
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
si prega di inserire il nome utente:asdfghjfghjdfghsgfghgsrtdgfhdk
Nome utente inserito: asdfghjfghjdfghsgfghgsrtdgfhdk
┌──(kali㉿kali)-[~/Desktop]
└─$


Minimize all open windows and show the desktop
kali@kali: ~/Desktop
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ cd Desktop
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
si prega di inserire il nome utente:1111111111111111111211111111
Nome utente inserito: 1111111111111111111211111111
┌──(kali㉿kali)-[~/Desktop]
└─$



1. Descrizione del quesito

Durante l'analisi del codice sorgente BOF.c, è stata valutata l'efficacia di una modifica strutturale: l'aumento della dimensione del buffer da **10 a 30 caratteri** per prevenire attacchi di tipo Buffer Overflow.

2. Analisi Tecnica: Perché la modifica è insufficiente

L'analisi dimostra che aumentare la dimensione del buffer **non elimina la vulnerabilità**. Il problema fondamentale non è quanto spazio sia disponibile, ma come il programma gestisce l'inserimento dei dati.

- **Assenza di controllo (Bounds Checking):** Se il programma utilizza funzioni insicure come scanf() o gets(), continuerà a scrivere in memoria finché l'utente non preme "Invio", indipendentemente dal fatto che il buffer sia di 10, 30 o 100 caratteri.
- **Adattabilità dell'exploit:** Per un attaccante, cambiare un attacco da 11 caratteri a uno di 31 caratteri è un'operazione banale. Basta inviare una stringa di input più lunga ("Payload") per sovrascrivere comunque lo Stack.
- **Sovrascrittura dello Stack:** Una volta superato il limite dei 30 caratteri, i dati in eccesso andranno comunque a corrompere il *Saved Frame Pointer* e, soprattutto, il *Return Address* (l'indirizzo che dice alla CPU dove andare dopo la funzione).

3. Soluzione Definitiva: Il "Bounds Checking"

Per rendere il codice realmente sicuro, è necessario utilizzare funzioni che limitino fisicamente la quantità di dati letti in base allo spazio disponibile.

Confronto Codice:

Versione Vulnerabile (anche con buffer a 30):

C

```
char buffer[30];

printf("Inserisci nome: ");

scanf("%s", buffer); // PERICOLOSO: non si ferma a 30.
```

Versione Sicura (Soluzione corretta):

C

```
char buffer[30];

printf("Inserisci nome: ");

// fgets(variabile, dimensione_massima, sorgente)

fgets(buffer, 30, stdin); // SICURO: taglia l'input al 29esimo carattere (+ \0).
```

4. Conclusioni

La sicurezza informatica non si ottiene aumentando le dimensioni dei contenitori (Security by Obscurity), ma implementando controlli rigorosi sugli input. La sostituzione di scanf con fgets garantisce che la memoria del programma rimanga integra, impedendo qualsiasi tentativo di Buffer Overflow